05 — NLP & Text Processing

Time: ~4-5 hours | **Level**: Intermediate

What you'll learn:

- Why text is hard for computers (it's not numbers!)
- Tokenisation: splitting text into pieces
- Bag of Words and TF-IDF: classic text representations
- Word Embeddings: Word2Vec, why they revolutionised NLP
- Text classification pipeline end-to-end
- How this connects to LLM tokenisers (BPE, SentencePiece)

Prerequisites: Notebooks 01-04

Why This Matters for Fine-Tuning

Before you can fine-tune an LLM, you need to understand how text becomes numbers. Every `model.generate(input_ids)` call you'll see in Notebooks 08-10 relies on tokenisation.

In [ ]:
```
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter
import re

# We'll create our own mental health-themed text data throughout this notebo
documents = [
    "Patient reports persistent feelings of sadness and hopelessness for two
    "Experiencing severe anxiety attacks with rapid heartbeat and shortness
    "Sleep patterns disrupted with early morning awakening and fatigue throu
    "Patient describes loss of interest in previously enjoyed activities and
    "Reports racing thoughts difficulty concentrating and elevated mood last
    "Feeling overwhelmed by daily tasks and experiencing frequent crying epi
    "Patient shows signs of improvement after medication adjustment and ther
    "Describes intrusive thoughts and compulsive behaviors interfering with
    "Reports improved sleep quality and increased energy levels after treatm
    "Experiencing panic disorder with agoraphobia avoiding public spaces and
]

labels = ['depression', 'anxiety', 'depression', 'depression', 'bipolar',
          'depression', 'improving', 'ocd', 'improving', 'anxiety']

print(f'Dataset: {len(documents)} clinical notes')
print(f'Labels: {Counter(labels)}')
print(f'\nExample: "{documents[0]}"')
print(f'Label: {labels[0]}')
```

1. Tokenisation — Splitting Text into Pieces

A **token** is the smallest unit of text the model works with.

Three levels of tokenisation:

1. **Word-level**: `"I feel sad"` → `["I", "feel", "sad"]`
2. **Subword-level**: `"unhappiness"` → `["un", "happi", "ness"]` (used by modern LLMs)
3. **Character-level**: `"sad"` → `["s", "a", "d"]`

Why subwords?

- Word-level: vocabulary is HUGE, can't handle new words ("COVID-19" in 2020)
- Character-level: sequences are very long, hard to learn meaning
- Subword: best of both — manageable vocab, handles new words

In [ ]:
```
# ── Word-level tokenisation (simple) ──────────────────────────

def simple_tokenise(text):
    """Lowercase and split on non-alphanumeric characters."""
    return re.findall(r'[a-z]+', text.lower())

# Tokenise all documents
tokenised = [simple_tokenise(doc) for doc in documents]

print('Document 0 tokens:')
print(tokenised[0])
print(f'Number of tokens: {len(tokenised[0])}')

# Build vocabulary (all unique words)
all_tokens = [t for doc in tokenised for t in doc]
vocab = sorted(set(all_tokens))
word2idx = {word: i for i, word in enumerate(vocab)}

print(f'\nTotal unique words (vocabulary size): {len(vocab)}')
print(f'Total tokens across all docs: {len(all_tokens)}')
print(f'\nFirst 15 words in vocab: {vocab[:15]}')
print(f'\n💡 Real LLM vocabularies: GPT-2=50,257  |  Phi-3=32,064  |  LLaMA-
```

2. Bag of Words (BoW)

The simplest text representation: count how many times each word appears.

"Patient reports sadness" → `[0, 0, ..., 1, ..., 1, ..., 1, ..., 0]`

Ignores word ORDER — "dog bites man" = "man bites dog" (a known limitation).

In [ ]:
```python
# ── Bag of Words from scratch ──────────────────────────────────

def bag_of_words(tokenised_doc, vocab_dict):
    """Convert a tokenised document into a count vector."""
    vec = np.zeros(len(vocab_dict))
    for token in tokenised_doc:
        if token in vocab_dict:
            vec[vocab_dict[token]] += 1
    return vec

# Create BoW matrix
bow_matrix = np.array([bag_of_words(doc, word2idx) for doc in tokenised])
print(f'BoW matrix shape: {bow_matrix.shape}')  # (10 docs, vocab_size)
print(f'Each document is a {len(vocab)}-dimensional vector')

# Show top words in first document
doc0_counts = bow_matrix[0]
top_indices = doc0_counts.argsort()[::-1][:5]
print(f'\nDoc 0: "{documents[0]}"')
print(f'Top words: {[(vocab[i], int(doc0_counts[i])) for i in top_indices]}'

print(f'\n⚠️  Limitation: BoW treats "patient is happy" and "happy is patier
print('   It also gives equal weight to common words like "the" and rare wor
```

3. TF-IDF — Smarter Word Weighting

TF-IDF (Term Frequency–Inverse Document Frequency) solves the "common words" problem.

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

Where:

- $\text{TF}(t, d) = \frac{\text{count of term } t \text{ in doc } d}{\text{total terms in doc } d}$ (how often this word appears)
- $\text{IDF}(t) = \log \frac{N}{\text{docs containing } t}$ (how rare this word is across all docs)

Intuition: Words that appear frequently in ONE document but rarely across ALL documents are the most informative.

In [ ]:
```python
# ── TF-IDF from scratch ──────────────────────────────────

def compute_tfidf(tokenised_docs, vocab_dict):
    """Build a TF-IDF matrix from scratch."""
    n_docs = len(tokenised_docs)
    vocab_size = len(vocab_dict)

    tf_matrix = np.zeros((n_docs, vocab_size))
    df_counts = np.zeros(vocab_size)  # document frequency

    for i, doc in enumerate(tokenised_docs):
        counts = Counter(doc)
```

```
        doc_len = len(doc)
        for word, count in counts.items():
            if word in vocab_dict:
                idx = vocab_dict[word]
                tf_matrix[i, idx] = count / doc_len  # TF
                df_counts[idx] += 1                  # count docs

    # IDF: log(N / df) — add 1 to avoid division by zero
    idf = np.log(n_docs / (df_counts + 1))

    tfidf = tf_matrix * idf  # element-wise multiply
    return tfidf

tfidf_matrix = compute_tfidf(tokenised, word2idx)
print(f'TF-IDF matrix shape: {tfidf_matrix.shape}')

# Compare BoW vs TF-IDF for document 0
print(f'\nDoc 0: "{documents[0]}"')
bow_top = bow_matrix[0].argsort()[::-1][:5]
tfidf_top = tfidf_matrix[0].argsort()[::-1][:5]

print(f'\nBoW top words:    {[vocab[i] for i in bow_top]}')
print(f'TF-IDF top words: {[vocab[i] for i in tfidf_top]}')
print(f'\n💡 TF-IDF promotes rare, distinctive words over common ones.')
```

4. Word Embeddings — Words as Vectors

BoW and TF-IDF have a fatal flaw: they don't understand **meaning**.

- "happy" and "joyful" are as different as "happy" and "car"
- Every word is an independent dimension

Word embeddings solve this by mapping words to **dense vectors** where similar words are close together:

```
"king"  → [0.2, 0.8, -0.1, 0.5, ...]  (300 dimensions)
"queen" → [0.3, 0.7, -0.2, 0.4, ...]  ← similar!
"car"   → [-0.5, 0.1, 0.9, -0.3, ...]  ← very different
```

Famous result from Word2Vec (2013):

$$\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$$

In [ ]:
```
# ── Understanding embeddings with a toy example ──────────────────
import torch
import torch.nn as nn

# PyTorch's Embedding layer: lookup table of learnable vectors
vocab_size = len(vocab)
embedding_dim = 16  # each word → 16-dimensional vector (real models use 300
```

```
embedding = nn.Embedding(vocab_size, embedding_dim)

# Look up a word
word = 'anxiety'
if word in word2idx:
    idx = torch.tensor([word2idx[word]])
    vec = embedding(idx)
    print(f'"{word}" → index {word2idx[word]} → embedding shape: {vec.shape}')
    print(f'Vector (first 8 dims): {vec[0, :8].detach().numpy().round(3)}')

# Look up a sentence
sentence = 'patient reports anxiety'
tokens = simple_tokenise(sentence)
indices = torch.tensor([word2idx[t] for t in tokens if t in word2idx])
vectors = embedding(indices)  # (n_tokens, embedding_dim)
print(f'\nSentence: "{sentence}"')
print(f'Token indices: {indices.tolist()}')
print(f'Embedded shape: {vectors.shape}')  # (3, 16)

print(f'\n💡 These embeddings start RANDOM and are LEARNED during training.')
print('   After training, similar words will have similar vectors.')
print('   This is exactly what happens inside GPT, BERT, and Phi-3.')
```

5. Text Classification Pipeline

Let's put it all together: classify mental health notes using embeddings + a neural network.

In [ ]:
```
# ── Complete text classification model ──────────────────────────
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from collections import Counter

# Larger synthetic dataset for training
np.random.seed(42)
templates = {
    'depression': [
        'feeling sad hopeless and tired all day',
        'lost interest in activities social withdrawal',
        'persistent sadness and low energy levels',
        'difficulty sleeping and loss of appetite',
        'feeling worthless and unable to concentrate',
    ],
    'anxiety': [
        'panic attacks with rapid heartbeat sweating',
        'constant worry about everything feeling restless',
        'fear of social situations avoiding crowds',
        'racing thoughts and difficulty relaxing',
        'physical tension headaches and nervousness',
    ],
}

# Generate 200 samples with random word substitutions
train_texts, train_labels = [], []
```

```
for _ in range(100):
    for label, temps in templates.items():
        text = np.random.choice(temps)
        # Add some noise: randomly drop/duplicate words
        words = text.split()
        if np.random.random() > 0.5:
            words.append(np.random.choice(words))
        np.random.shuffle(words)
        train_texts.append(' '.join(words))
        train_labels.append(0 if label == 'depression' else 1)

print(f'Generated {len(train_texts)} training samples')
print(f'Label distribution: {Counter(train_labels)}')

# Build vocabulary from training data
all_words = [w for t in train_texts for w in simple_tokenise(t)]
train_vocab = sorted(set(all_words))
train_w2i = {w: i+1 for i, w in enumerate(train_vocab)}  # 0 reserved for pa
train_w2i['<PAD>'] = 0

print(f'Vocabulary size: {len(train_w2i)}')

# Convert texts to padded index sequences
max_len = 12
def text_to_indices(text, w2i, max_len):
    tokens = simple_tokenise(text)
    indices = [w2i.get(t, 0) for t in tokens[:max_len]]
    indices += [0] * (max_len - len(indices))  # pad
    return indices

X_idx = torch.tensor([text_to_indices(t, train_w2i, max_len) for t in train_
y_cls = torch.tensor(train_labels, dtype=torch.float32).unsqueeze(1)

print(f'Input shape: {X_idx.shape}')  # (200, 12)
print(f'Example indices: {X_idx[0].tolist()[:8]}...')
```

In [ ]:

```
# ── Text Classifier: Embedding → Average → Linear ──────────────

class TextClassifier(nn.Module):
    """
    Simple but effective text classifier:
    1. Embed each token (index → vector)
    2. Average all token vectors (sentence representation)
    3. Feed through linear layers for classification

    This is a simplified version of what BERT does.
    """
    def __init__(self, vocab_size, embed_dim, hidden_dim, n_classes=1):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.classifier = nn.Sequential(
            nn.Linear(embed_dim, hidden_dim),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(hidden_dim, n_classes),
            nn.Sigmoid()
```

```
        )

    def forward(self, x):
        # x: (batch, seq_len) — indices
        embedded = self.embedding(x)         # (batch, seq_len, embed_dim)

        # Average pooling (ignore padding)
        mask = (x != 0).unsqueeze(-1).float()  # (batch, seq_len, 1)
        summed = (embedded * mask).sum(dim=1)  # (batch, embed_dim)
        lengths = mask.sum(dim=1).clamp(min=1) # avoid division by zero
        averaged = summed / lengths              # (batch, embed_dim)

        return self.classifier(averaged)        # (batch, 1)

# Train
torch.manual_seed(42)
clf = TextClassifier(len(train_w2i), embed_dim=32, hidden_dim=16)
optimizer = optim.Adam(clf.parameters(), lr=0.005)
loss_fn = nn.BCELoss()

loader = DataLoader(TensorDataset(X_idx, y_cls), batch_size=32, shuffle=True

for epoch in range(50):
    total_loss = 0
    for bx, by in loader:
        pred = clf(bx)
        loss = loss_fn(pred, by)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    if epoch % 10 == 0:
        with torch.no_grad():
            all_pred = clf(X_idx)
            acc = ((all_pred > 0.5).float() == y_cls).float().mean()
        print(f'Epoch {epoch:2d} | Loss: {total_loss/len(loader):.4f} | Acc:

print('\n✅ Text classifier trained!')
```

6. From Word Embeddings to LLM Tokenisers

Modern LLMs don't use word-level tokenisation. They use **subword tokenisation**:

Method	Used By	How It Works
BPE (Byte Pair Encoding)	GPT-2, GPT-3, GPT-4	Iteratively merges frequent character pairs
WordPiece	BERT	Similar to BPE, uses likelihood instead of frequency
SentencePiece	LLaMA, T5, Phi-3	Language-agnostic, treats text as raw bytes

BPE Example:

```
"unhappiness" → BPE → ["un", "happi", "ness"]
"COVID-19"    → BPE → ["COVID", "-", "19"]  (handles novel
words!)
```

Why subwords matter for fine-tuning:

When you see `tokenizer("Patient reports anxiety")` in Notebook 08, it's doing BPE/SentencePiece — not the simple `split()` we used above.

In [ ]:

```python
# ─── Simplified BPE demonstration ──────────────────────────────

def simple_bpe_demo(text, num_merges=10):
    """
    Demonstrate the core idea of Byte Pair Encoding.

    BPE starts with individual characters, then repeatedly
    merges the most frequent adjacent pair.
    """
    # Start with characters (add end-of-word marker)
    words = text.lower().split()
    word_freqs = Counter(words)

    # Split each word into characters
    splits = {word: list(word) for word in word_freqs}

    print(f'Initial tokens: {list(set(c for word in splits.values() for c in
    print(f'\nPerforming {num_merges} merges:')

    for i in range(num_merges):
        # Count all adjacent pairs
        pair_counts = Counter()
        for word, freq in word_freqs.items():
            chars = splits[word]
            for j in range(len(chars) - 1):
                pair_counts[(chars[j], chars[j+1])] += freq

        if not pair_counts:
            break

        # Find most frequent pair
        best_pair = pair_counts.most_common(1)[0]
        pair, count = best_pair
        merged = pair[0] + pair[1]

        # Merge this pair in all words
        for word in splits:
            chars = splits[word]
            new_chars = []
            j = 0
            while j < len(chars):
                if j < len(chars) - 1 and chars[j] == pair[0] and chars[j+1]
                    new_chars.append(merged)
                    j += 2
                else:
```

```
                    new_chars.append(chars[j])
                    j += 1
            splits[word] = new_chars

        print(f'  Merge {i+1}: "{pair[0]}" + "{pair[1]}" → "{merged}" (count

    print(f'\nFinal tokenisation:')
    for word, tokens in sorted(splits.items()):
        print(f'  "{word}" → {tokens}')

simple_bpe_demo('patient reports feeling sad and hopeless patient feeling ar

print('\n💡 This is exactly how GPT\'s tokeniser was built!')
print('   The actual tokeniser runs on a massive corpus (all of the internet
print('   Result: common words stay whole, rare words get split into subworc
```

✅ Key Takeaways

1. **Tokenisation** converts text to numbers — the fundamental first step
2. **Bag of Words** counts word frequencies — simple but ignores meaning and order
3. **TF-IDF** improves on BoW by weighting rare, distinctive words higher
4. **Word Embeddings** map words to dense vectors where similar words cluster together
5. **Subword tokenisation** (BPE) is what modern LLMs actually use
6. The pipeline: Text → Tokenise → Embed → Encode → Classify/Generate

What's Missing?

All methods so far either:

- Ignore word order (BoW, TF-IDF)
- Average away sequence information (our classifier)

Next: 06 — RNNs & Sequence Models — models that understand order and context